

Core Java & Spring Boot Interview Study Guide

Part 1: Core Java & Object Class Methods

Q: Which methods are present in the Object class?

Answer: The Object class is the root class in Java. The primary methods include the `<u>equals()` method, `<u>hashCode()` method, `<u>toString()` method, `<u>getClass()`, `<u>clone()`, `<u>finalize()`, `<u>wait()`, `<u>notify()`, and `<u>notifyAll()`.

Q: Explain the equals() method.

Answer: By default, the `equals()` method compares the memory references of two objects. However, if we want to compare the actual content or state of the objects, we need to override the `equals()` method in our custom class.

Q: Explain the hashCode() method.

Answer: The JVM generates a unique integer value for each object, which is used for hashing based collections. When we call the `hashCode()` method, it returns this integer representation (hash code).

Q: Explain the toString() method.

Answer: The `toString()` method returns a string representation of the object. By default, it prints the class name followed by the @ symbol and the object's hash code in hexadecimal format. It is usually overridden to print the actual data inside the object.

Q: What is the contract between the hashCode() and equals() methods?

Answer: The strict contract is: If two objects are equal according to the `equals()` method, they must have the same hash code. However, if two objects have the same hash code, they are not strictly required to be equal (this leads to a hash collision).

Q: What happens if you override equals() but not hashCode()?

Answer: If you use a hashing-based collection like `HashMap`, and you override `equals()` but not `hashCode()`, equal objects will generate different hash codes. They will end up in different buckets. Because of this, the `HashMap` won't realize the keys are the same, resulting in duplicate keys instead of updating the existing key-value pair.

Q: What happens if you override hashCode() but not equals()?

Answer: The objects will generate the same hash code and go to the same bucket in a HashMap. However, because equals() is not overridden, the map will treat them as different objects and chain them together in that bucket. The duplicate will not be overwritten or removed.

Q: How do you remove duplicates from a list of custom objects?

Answer: To remove duplicates, you pass the custom objects into a Set interface (like HashSet). For the Set to successfully identify and remove the duplicates, you must override both equals() and hashCode() in your custom object class.

Q: How do you sort custom objects?

Answer: Collections like TreeSet or TreeMap naturally sort wrapper classes. But for custom objects (like an Employee class), you must provide the sorting logic manually by implementing either the `Comparable` interface (for a single natural sorting sequence) or the `Comparator` interface (for multiple custom sorting sequences).

Part 2: Collections Framework & Multithreading

Q: What is the Collection framework in Java?

Answer: Collection is a root interface in the framework hierarchy. It has multiple sub-interfaces like `List`, `Set`, and `Queue`.

- **List** implementations: `ArrayList`, `LinkedList`, `Vector`.
- **Set** implementations: `HashSet`, `LinkedHashSet`, `TreeSet`.

Q: What is the difference between standard Collections and Concurrent Collections?

Answer: Standard collections are fail-fast; if a collection is modified while a thread is iterating over it, it throws a `ConcurrentModificationException`.

Concurrent collections are fail-safe. They don't throw this exception because they operate on a cloned copy of the data or use advanced locking mechanisms. For instance, `ConcurrentHashMap` uses segment locking (or bucket-level locking), allowing multiple threads to read and write safely and concurrently without locking the entire map, which drastically improves performance. Examples include `ConcurrentHashMap` and `CopyOnWriteArrayList`.

Q: What is a thread and how do you create one?

Answer: A thread is the smallest unit of execution within a process. Multithreading allows us to perform multiple independent tasks simultaneously. You can create a thread by:

1. Extending the `Thread` class
2. Implementing the `Runnable` interface
3. Implementing the `Callable` interface
4. Using the `ExecutorService` (Thread Pools)

Q: What is the difference between ArrayList and LinkedList?

Answer: * **ArrayList:** Stores elements in contiguous memory locations. Fetching data via an index is extremely fast ($O(1)$), making it preferable for frequent reading operations. However, insertion or deletion in the middle is costly because it requires shifting subsequent elements.

- **LinkedList:** Stores elements as nodes (data + pointers). To fetch an element, it must traverse the nodes ($O(n)$). However, it is highly preferable for frequent insertion and deletion because it only requires updating the node references, with no element shifting required.

Q: What is the difference between Set and Map?

Answer: * **Set:** Stores single elements and does not allow duplicate values. Examples: HashSet (no order), LinkedHashSet (insertion order), TreeSet (sorted order).

- **Map:** Stores elements in key-value pairs. It does not allow duplicate keys, but values can be duplicated. Examples: HashMap (no order), LinkedHashMap (insertion order), TreeMap (sorted by keys).

Q: Explain the internal working of a HashMap.

Answer: When you create a HashMap, it internally creates an array of nodes (called buckets) with a default capacity of 16.

When you use the `put()` method to add a key-value pair, the map calculates the hash code of the key. It then applies a mathematical formula against the array size to calculate the exact bucket index.

- If the bucket is empty, the key-value pair is stored there.
- If two different keys generate the same index (Hash Collision), the `equals()` method checks if the keys are identical. If they are, the old value is replaced.

If they are different, the new node is added to that bucket, forming a linked list (which turns into a balanced tree if the list gets too long).

Part 3: Spring Boot, JPA, & AOP

Q: What is JPA and how does it work?

Answer: JPA stands for Java Persistence API. It is a specification for managing relational data and performing database operations. In Spring Boot, you add the Spring Data JPA dependency and create an interface that extends `JpaRepository`. At runtime, Spring automatically generates the implementation (proxy) for standard database operations, saving you from writing boilerplate SQL.

Q: How do you connect a database to a Spring Boot application?

Answer: You configure the database credentials—such as the database URL, username, and password—inside the `application.yml` or `application.properties` file. Spring Boot's auto-configuration detects this and connects automatically.

Q: Is it possible to connect two different databases in one Spring Boot application?

Answer: Yes. Spring Boot auto-configures a single data source by default, but you can manually configure multiple databases by creating separate configuration classes. You would define two distinct `DataSource`, `EntityManagerFactory`, and `TransactionManager` beans for each database.

Q: What is Spring AOP?

Answer: Spring AOP stands for Aspect-Oriented Programming. It is used to separate cross-cutting concerns (secondary logic like logging, security, or transaction management) from the main business logic without modifying the core code.

- **Aspect:** The class containing the secondary logic.
- **Advice:** Specifies when the logic should run (e.g., `@Before`, `@After`, `@Around`).
- **Join Point:** A specific point during execution (like a method call) where advice can be applied.
- **Pointcut:** An expression that defines where (which specific methods/classes) the advice should be applied.
- **Weaving:** The process of linking the aspect to the main application code.

Q: What is a Transaction in Spring?

Answer: Spring Transactions manage database operations to ensure data integrity through Commit and Rollback mechanisms. If an operation spans multiple steps (like placing an order and processing payment) and one step fails, the transaction is rolled back to its original state so partial data isn't saved. It is configured using two main levels:

- **Propagation:** Defines how a transaction behaves when a method is called inside another active transaction.
- **Isolation:** Defines how separated a transaction is from other concurrent transactions.

Part 4: Security (JWT)

Q: What is the difference between Authentication and Authorization?

Answer: * **Authentication:** Verifying the identity of a user (e.g., checking if the username and password match the database).

- **Authorization:** Checking the permissions or boundaries of an authenticated user (e.g., verifying if a standard user is trying to access an admin-only API).

Q: Explain the flow of JWT authentication.

Answer: 1. The user sends a login request with their credentials.

2. The server validates the credentials against the database.

3. Upon successful validation, the server generates a JWT token and sends it back to the client.

4. For all subsequent API requests, the client attaches this token in the Authorization header.

5. The server intercepts the request, validates the token's signature, and if valid, processes the request.

Q: What are the three parts of a JWT token? [Correction added for completeness]

Answer: A JWT is a string separated by dots into three parts:

1. **Header:** Contains the algorithm used for signing the token.
2. **Payload:** Contains the claims (data like user ID, roles, and expiration time).

3. **Signature:** A cryptographic hash used to verify that the token has not been tampered with.

Q: Where is the JWT token stored on the web browser side?

Answer: The token can be stored in the browser's Local Storage or Session Storage. However, for better security against Cross-Site Scripting (XSS) attacks, it is highly recommended to store the JWT in an HttpOnly Cookie.

Part 5: Design Patterns & Modern Java Features

Q: What is the difference between the Singleton and Template design patterns?

Answer: * **Singleton:** A creational design pattern that restricts the instantiation of a class to a single object. No matter how many times you request the object, you receive the same instance.

- **Template:** A behavioral design pattern used for code reusability. It uses an abstract class to define the core skeleton of an algorithm, leaving the specific implementation details to be overridden by child classes.

Q: What are the new features introduced in Java 8?

Answer: Key features include the Stream API, Lambda Expressions, Functional Interfaces, the Optional class, and Default Methods inside interfaces.

Q: What is the Stream API?

Answer: The Stream API is used to process sequences of elements from collections. It supports a pipeline of operations:

- **Intermediate Operations:** Used for transforming or filtering data (they return a new stream). Examples: `map()`, `filter()`.
- **Terminal Operations:** Used to produce a final result or side effect. Examples: `count()`, `min()`, `max()`, `collect()`.

Q: What is the difference between `map()` and `filter()` in the Stream API?

Answer: * **`filter()`:** Used to evaluate a condition and filter out elements that do not match. (e.g., extracting only even numbers). The number of output elements can be less than the input.

- **map():** Used to transform data. It performs an operation on every single element in the sequence (e.g., multiplying every number by 2). The number of output elements matches the input.

Q: What is a prominent feature of Java 17?

Answer: Sealed Classes. By using the sealed keyword on a class or interface, you can restrict which classes are allowed to extend or implement it. You define the allowed child classes using the permits keyword. This gives developers precise control over inheritance boundaries (e.g., restricting a Payment interface so only CreditCard, DebitCard, and UPI classes can implement it).

Comprehensive Technical Interview Blueprint: Java & Spring Boot

This document provides an expanded, in-depth analysis of the core concepts discussed during the technical mock interview. It transforms basic definitions into advanced technical explanations, addressing common misconceptions and detailing the underlying architecture of Java and Spring Boot.

I. Object-Oriented Programming (OOP) Fundamentals

While the four pillars of OOP (Encapsulation, Inheritance, Polymorphism, and Abstraction) are foundational, senior-level understanding requires knowing exactly how these concepts materialize in memory and architecture.

The Mechanics of Abstraction

A common stumbling block is explaining how abstraction practically hides implementation at the code level.

- **The Concept:** Abstraction separates the *contract* (what a system does) from the *implementation* (how it does it).*
- **Code-Level Reality:** When a controller relies on a `UserService` interface, the controller is completely oblivious to whether the underlying database is PostgreSQL, MongoDB, or a mocked in-memory list. The implementation details are hidden behind the interface. If `UserServiceImpl` is heavily modified or completely rewritten, the controller code does not need a single line changed. This decoupling is the true definition of "hiding implementation details."*

Abstract Classes vs. Interfaces

- **Interfaces:** Define contracts. From Java 8 onwards, interfaces can hold default and static methods, meaning they are no longer purely abstract. Variables are implicitly public static final. Interfaces are the mechanism used to circumvent Java's restriction on multiple inheritance via the `implements` keyword.*
- **Abstract Classes:** Used when classes share a core identity (an "is-a" relationship) and share common state or mutable fields. They can have constructors, which are called during the instantiation of their concrete subclasses to initialize the inherited state.*

II. Spring Boot Architecture & Inversion of Control

Inversion of Control (IoC)

IoC shifts the responsibility of object creation and lifecycle management from the developer (using the new keyword) to the Spring framework. This promotes the Open/Closed Principle and extreme loose coupling.

IoC Containers:

- 1. BeanFactory:*** The foundational container. It utilizes **lazy initialization**, meaning beans are only created when explicitly requested. It is highly memory-efficient but rarely used in modern enterprise applications.
- 2. ApplicationContext:*** The advanced container used by default in Spring Boot. It utilizes **eager initialization** for singletons, loading them during application startup. It also provides enterprise features like event publishing, internationalization (i18n), and AOP integration.

Dependency Injection (DI) Strategies

DI is the design pattern used to implement IoC.

<i>Injection Type</i>	<i>Description</i>	<i>Architectural Verdict</i>
<i>Constructor Injection</i>	<i>Dependencies are provided via the class constructor.</i>	<i>Best Practice.</i> It ensures the class cannot be instantiated without its dependencies, preventing NullPointerExceptions. It also allows fields to be declared as final, ensuring immutability.
<i>Setter Injection</i>	<i>Dependencies are provided via setter methods.</i>	<i>Situational.</i> Useful only when dependencies are truly optional or need to be reconfigured at runtime.
<i>Field Injection</i>	<i>Dependencies are injected directly into fields using @Autowired.</i>	<i>Anti-Pattern.</i> It relies on Java Reflection, which introduces performance overhead. It also tightly couples the class to the Spring DI container, making unit testing without Spring extremely difficult.

III. The Java Collections Framework

List vs. Set Dynamics

- **List:** An ordered collection that allows duplicates. Accessed via index.
 - *ArrayList:* Backed by a dynamic array. Excellent for random access ($O(1)$) but poor for middle insertions/deletions due to element shifting.
 - *Vector:* The legacy, synchronized equivalent of *ArrayList*. It is thread-safe but incurs massive performance overhead due to intrinsic locking on every method. Modern applications avoid *Vector*.
- **Set:** An unordered collection that guarantees uniqueness.
 - *HashSet:* Backed by a *HashMap*. No ordering.
 - *TreeSet:* Backed by a *TreeMap*. Maintains elements in sorted order based on their natural ordering or a provided custom comparator.

Custom Sorting: Comparable vs. Comparator

When storing custom objects (e.g., *Employee*) in a *TreeSet* or sorting them via *Collections.sort()*, Java needs to know how to sort them.

- **Comparable (java.lang):** Modifies the original class by implementing the *compareTo()* method. It dictates the natural or default sorting sequence.
- **Comparator (java.util):** Creates an external sorting logic class by implementing *compare()*. This is highly preferred because it adheres to the Open/Closed Principle—you can create multiple sorting strategies (e.g., sort by age, sort by salary) without touching the original *Employee* code.

IV. HashMap Internals & Advanced Hashing

Understanding the internal mechanics of a *HashMap* is a hallmark of a proficient Java developer.

The Insertion Flow:

1. **Index Calculation:** When *put(key, value)* is called, Java calculates the key's hash code. It then uses a bitwise AND operation $(n - 1) \& \text{hash}$ (where n is the array length) to determine the exact bucket index.
2. **Hash Collision:** If two distinct keys compute to the exact same bucket index, a collision occurs.

3. **Resolution:** The `HashMap` handles collisions by chaining elements in a `LinkedList` at that specific index.
4. **Equality Check:** If a key maps to a bucket that already contains nodes, the map uses the `equals()` method to check if the exact key already exists. If `equals()` returns `true`, the value is overwritten. If `false`, the new node is appended to the chain.
5. **Java 8 Optimization (Treeification):** If a single bucket's linked list grows beyond 8 elements (and the total map capacity is at least 64), the linked list transforms into a **Red-Black Tree**, improving the worst-case search time from $O(n)$ to $O(\log n)$.

Crucial Rule: If you use custom objects as map keys, you **must** override both `hashCode()` and `equals()`. If you do not, two objects with the exact same data will generate different hash codes (defaulting to memory addresses), resulting in memory leaks and lost data.

V. Concurrency, Synchronization, and Thread Safety

Solving the `ConcurrentModificationException`

This exception is thrown by **fail-fast iterators** (like those in `ArrayList` or `HashMap`) when they detect that the collection's structure was modified (an element was added or removed) while a thread was iterating over it.

- **The Solution:** Use **Concurrent Collections** (fail-safe iterators). For example, `CopyOnWriteArrayList` creates a fresh, cloned copy of the underlying array for every write operation, allowing iteration over the original copy to continue safely without locking the readers.

Thread Creation & Callable

While extending `Thread` or implementing `Runnable` are standard, they have a critical limitation: the `run()` method returns `void` and cannot throw checked exceptions.

- **Callable Interface:** Introduced in Java 5, `Callable`'s `call()` method can return a result and throw checked exceptions. When submitted to an `ExecutorService`, it returns a `Future` object, which acts as a placeholder for the result that will eventually be computed.

The Problem with the `synchronized` Keyword

Using the `synchronized` keyword (either on a method or a block) enforces an intrinsic lock (monitor lock). Only one thread can enter the synchronized area at a time.

- **The Disadvantage:** It can lead to severe performance bottlenecks and thread starvation. If a thread gets stuck inside a synchronized block, all other waiting threads are blocked indefinitely.
- **The Modern Solution: `ReentrantLock`** (from `java.util.concurrent.locks`). It provides explicit locking mechanisms. It offers advanced features like `tryLock()` (which attempts to acquire the lock but moves on if it's unavailable, preventing deadlocks) and fairness policies (ensuring the longest-waiting thread gets the lock next).

VI. Spring Data JPA & Transaction Management

Beyond Standard Repositories

While `JpaRepository` provides out-of-the-box methods (`save()`, `findAll()`), real-world applications require complex data retrieval.

- **The `@Query` Annotation:** Allows developers to write custom queries directly above repository methods. By default, these are written in **JPQL** (Java Persistence Query Language), which queries the entity objects rather than the database tables. You can also set `nativeQuery = true` to write raw, database-specific SQL.

Spring Transactions (`@Transactional`)

Transactions ensure that multiple database operations are treated as a single, atomic unit of work—they either completely succeed or completely roll back. If a payment is saved but the order creation throws a `RuntimeException`, the `@Transactional` proxy automatically issues a rollback command to the database, wiping out the orphaned payment record.

Core Transaction Configurations:

- **Propagation:** Defines how transactions interact with existing ones.
 - **REQUIRED (default):** Joins an existing transaction if one exists; otherwise, creates a new one.
 - **REQUIRES_NEW:** Suspends the current transaction and forces the creation of a brand new, independent transaction.

- **Isolation:** Defines how data visibility is handled between concurrent transactions, preventing issues like Dirty Reads (reading uncommitted data) or Phantom Reads. Levels range from `READ_UNCOMMITTED` to the highly restrictive `SERIALIZABLE`.

VII. Java 8 Masterclass: Streams & Lambdas

Functional Interfaces & Lambdas

A Functional Interface is an interface containing exactly one abstract method (annotated with `@FunctionalInterface`). Lambda expressions `() -> {}` provide a concise, anonymous implementation of that single method, drastically reducing boilerplate code.

The Stream API

Streams act as a declarative pipeline to process sequences of data efficiently.

1. **Intermediate Operations:** Return a new Stream, allowing operations to be chained. They are *lazy*, meaning they do not execute until a terminal operation is invoked.
 - `filter(Predicate)`: Removes elements that do not match a given condition.
 - `map(Function)`: Transforms data on a 1-to-1 basis (e.g., extracting the `getName()` string from an `Employee` object).
 - `flatMap()`: Transforms and flattens nested data structures. If you have a `List<List<String>>`, `flatMap` will merge them into a single, flat `List<String>`.
2. **Terminal Operations:** Trigger the processing pipeline and produce a final result (e.g., `collect()`, `count()`, `reduce()`).

Parallel Streams: By calling `parallelStream()`, Java automatically splits the data into chunks and processes them simultaneously using the common `ForkJoinPool`. While powerful, it should be used cautiously; for small datasets or stateful operations, the overhead of managing multiple threads can actually make parallel streams slower than sequential ones.